UNIVERSIDADE ESTADUAL DE MATO GROSSO DO SUL
UNIDADE UNIVERSITÁRIA DE DOURADOS
CURSO DE CIÊNCIA DA COMPUTAÇÃO
DISCIPLINA : ANÁLISE DE ALGORITMOS
P2-2025

ACADÊMICO(A): Dizemonte D. Franco RGM 50575

```
algoritmo bubblesort
  Entrada: vetor A com n inteiros
  Saída: vetor A ordenado

1   para i ← 1 até n-1 passo 1
2     para j ← 2 até n passo 1
3       se A[ j-1 ] < A[ j ] então
4         aux ← A[ j ]
5         A[j] ← A[ j – 1 ]
6         A[ j – 1 ] ← aux
7       fim_se
8     fim_para
9   fim_para
fim_algoritmo
```

```
algoritmo busca
  Entrada: vetor A com n inteiros e;
           um inteiro K
  Saída : a posição de K ou -1.

1   i ← 1
2   enquanto i ≤ n e A[i] ≠ K faça
3       i = i+1
4   fim_enquanto
5   se i>0 e A[i] = K então
6     retorne( i )
7   senão
8     retorne( -1 )
9   fim_se
fim_algoritmo
```

1) Baseando-se numa hipótese, construa um algoritmo para determinar a posição do menor elemento em um vetor de tamanho n de inteiros. Defina sua hipótese. 3,0

2) Determine o Invariante do Laço 3-8 do algoritmo bubblesort. 2,0

3) Prove a corretude do algoritmo busca baseando-se no invariante do laço:
"O elemento K não está no subvetor A[1..i-1]." 3,0

4) Construa um algoritmo que implemente a busca binária utilizando Divisão e Conquista. 2,0

Lizandro D. França 50575 8.5 (oito e meio)

2.5 1. Hipótese: Queremos "encontrar" a posição do menor elemento num vetor A[1..n-1]

```
algoritmo encontraMenor(A, n)
    se n == 1 então
        retorne 1
    fim se
    menor = encontraMenor(A, n-1)
    se A[n] < A[menor] então
        menor = n
    fim se
    retorne menor
fim algoritmo
```

2.0 2. O menor elemento do subvetor A[i..j-1] está na última posição do subvetor A[i..j-1].

2.5 3. INICIALIZAÇÃO Antes da 1ª iteração: Inicialmente, a variável "i" possui valor 1, então o subvetor A[1..1-1], ou seja, A[1..0] é vazio e por isso é óbvio que o elemento K não está no subvetor A[0], portanto a invariante é válida.

MANUTENÇÃO Durante as iterações: Durante, é realizada uma comparação se "i" é menor que o tamanho do vetor e se o elemento na posição "i" não é o elemento K procurado, caso as condições foram atendidas, o "i" é incrementado em 1, observe que o "i" só incrementa quando A[i] ≠ K, ou seja, ele garante que os incrementos ocorreram apenas se o elemento K não está no subvetor A[1..i-1], pois caso a condição A[i] ≠ K não for atendida, quer dizer que o elemento foi encontrado no índice "i", não precisando mais de incrementar, logo, a invariante permanece verdadeira.

Finalização: Garantido que no subvetor A[1..i-1] não contém K, resta verificar se a posição "i" contém o elemento K procurado, o que

é realizado na comparação da linha 5, onde i sempre será maior que 0 desde que sua inicialização é feita "i←1" (linha 1), verificando se A[i] = K, caso positivo o algoritmo retorna o índice onde K foi encontrado, caso contrário -1 indicando que K não está no vetor, ainda mantendo a invariante válida.

1.5 4.

```
algoritmo buscaBinária(A, ini, fim, x)
    meio = (fim + ini)/2
    se (ini - fim) == 0 então
        se A[ini] == x então
            retorno ini
        senão
            retorne -1
        fim se
    senão
        se A[meio] == x então
            retorno meio
        senão
            se X < A[meio] então
                X1 = buscaBinária(A, ini, meio, x)
            senão
                X2 = buscaBinária(A, meio+1, fim, x)
            fim se
        fim se
    fim se
    se X1 ≠ -1 então
        retorno X1
    senão
        se X2 ≠ -1 então
            retorno X2
        senão
            retorne -1
        fim se
    fim se
fim algoritmo
```

retorne(X1)